

Custom ByT5-Inspired T5 Model
Complete Architecture Explanation with Diagrams
Gondi ↔ English / Hindi Translation | ~25.5M Parameters

What Is This Model?

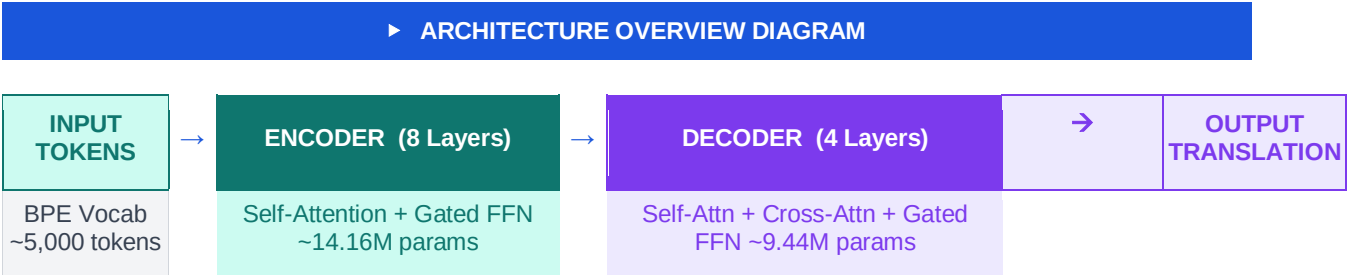
This document explains a custom multilingual neural machine translation model designed for Gondi ↔ English/Hindi translation. It draws inspiration from two Google Research architectures:

- T5** — T5 (Text-to-Text Transfer Transformer): a powerful encoder-decoder sequence-to-sequence model.
- ByT5** — ByT5 (Byte-Level T5): a variant that adopts an asymmetric encoder/decoder design, using a deeper encoder and shallower decoder.

The result is a lightweight (~25.5M parameter) model tailored for a low-resource language (Gondi) trained on only ~60,000 sentence pairs, efficient enough to run on a consumer RTX GPU.

1. High-Level Architecture Overview

The model follows a classic Encoder–Decoder (Seq2Seq) structure. Source tokens are processed by the encoder to produce rich contextual representations, which the decoder then attends to while generating the target translation one token at a time.



Key Asymmetry (ByT5 Inspiration): 8 encoder layers vs 4 decoder layers.
This is intentional — understanding the source language deeply is harder than generating the target.
A heavier encoder means better semantic compression, while a lighter decoder speeds up auto-regressive generation.

2. Core Configuration Table

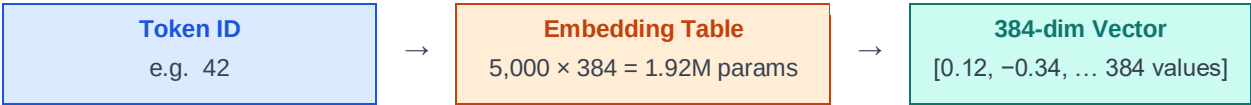
Every hyperparameter below was chosen to balance model expressiveness with the constraints of a 60K training corpus and consumer GPU memory.

Parameter	Value	Design Rationale
Vocabulary Size	~5,000	Custom BPE — covers Gondi, Hindi & English subwords efficiently
d_model	384	Embedding dimension; balanced between expressiveness and memory
d_ff	1,024	FFN inner size $\approx 2.67 \times d_model$; standard T5 ratio

num_heads	6	$6 \times 64 = 384$; each head attends independently in parallel
d_kv	64	Per-head key/query/value dimensionality
Encoder Layers	8	Deep encoder improves source-language semantic understanding
Decoder Layers	4	Shallow decoder speeds up auto-regressive generation
Activation	Gated-GeLU	Outperforms ReLU on morphology-rich / low-resource tasks
Total Params	~25.5M	Fits on consumer RTX GPU; avoids overfitting on 60K sentence pairs

3. Embedding Layer

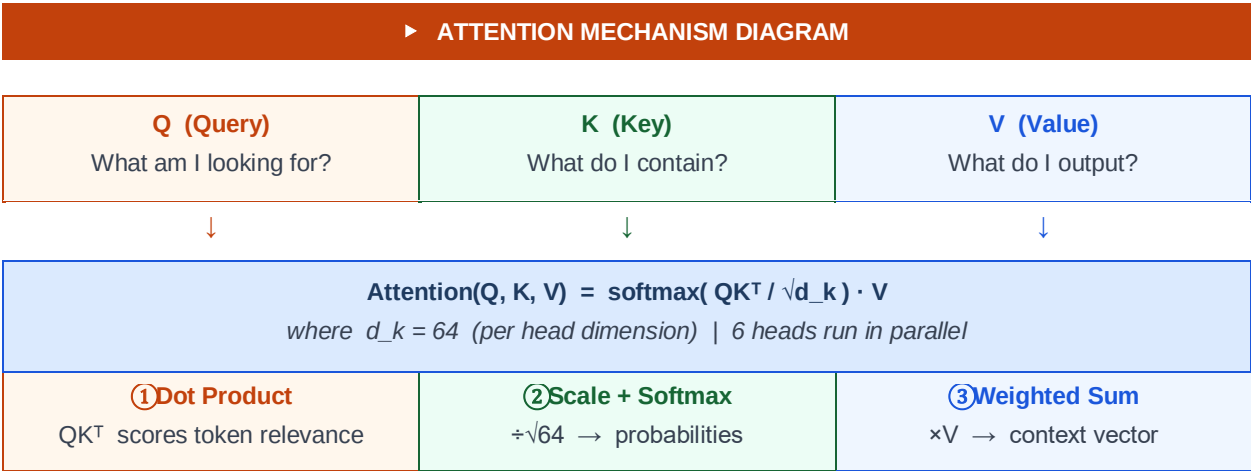
Before any processing, input token IDs are looked up in an embedding table that maps each ID to a dense vector.



Formula: $\text{vocab_size} \times \text{d_model} = 5,000 \times 384 = 1,920,000 \approx 1.92\text{M parameters}$

4. Multi-Head Self-Attention

Self-attention allows every token to look at every other token in the sequence and decide how much attention to give each. Running this in parallel across multiple 'heads' lets the model capture different types of relationships simultaneously.

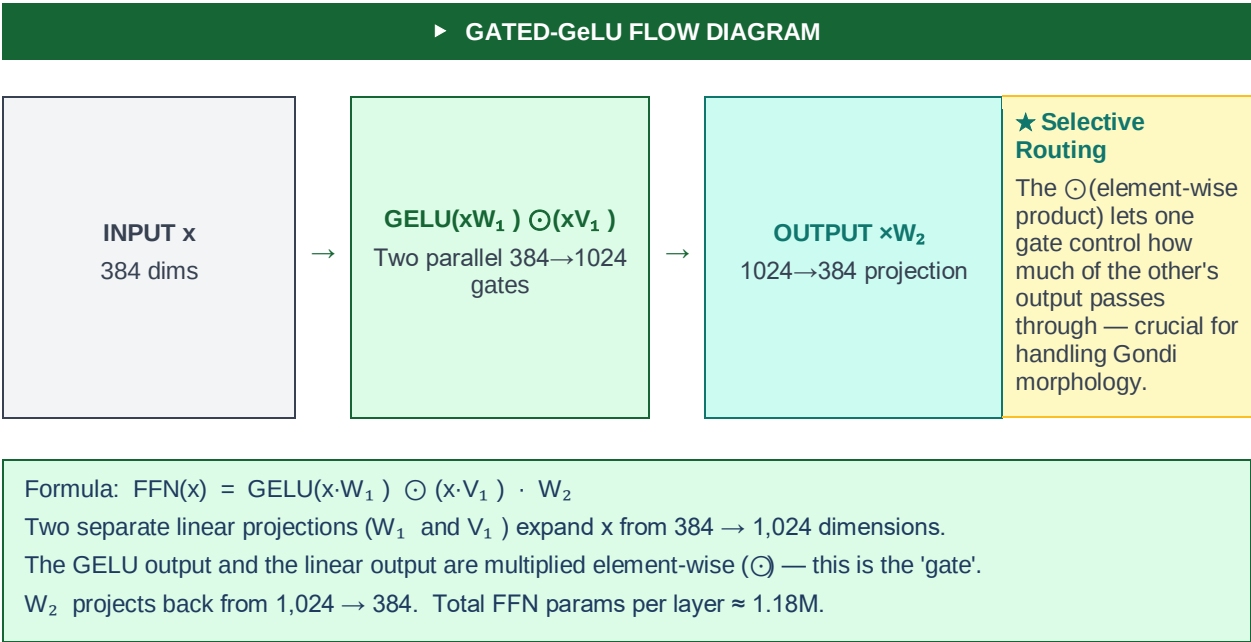


Why 6 Heads × 64 Dimensions?

- Total attention width = $6 \times 64 = 384$, matching `d_model` exactly.
- Each head can specialize: one head may capture subject-verb agreement, another captures negation, etc.
- For Gondi (a Dravidian-family language with complex morphology), multiple heads help capture case markers and postpositions.

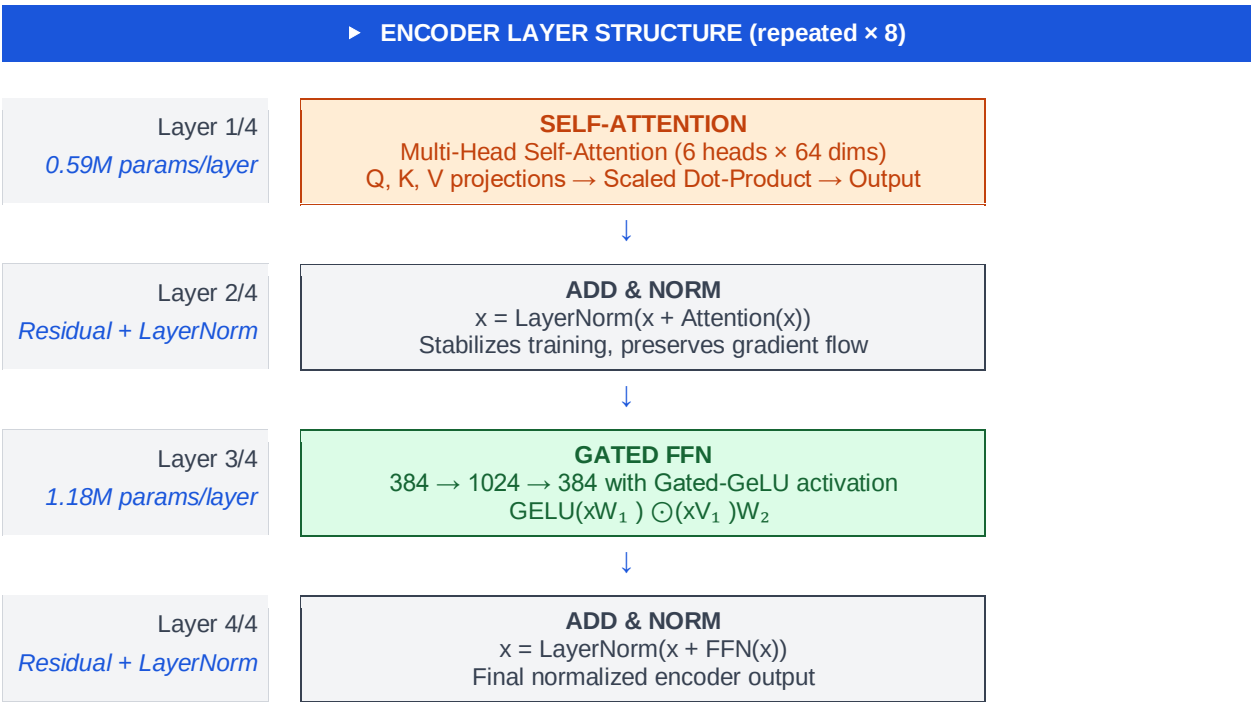
5. Feed-Forward Network with Gated-GeLU

After each attention sublayer, a Feed-Forward Network (FFN) independently processes each token position. This model uses Gated-GeLU instead of standard ReLU, which significantly improves performance on morphologically rich and low-resource languages.



6. Encoder Architecture (8 Layers)

The encoder reads all source tokens simultaneously and builds rich contextual representations. Each of the 8 encoder layers applies the same two sublayers sequentially, with residual connections and layer normalization after each.



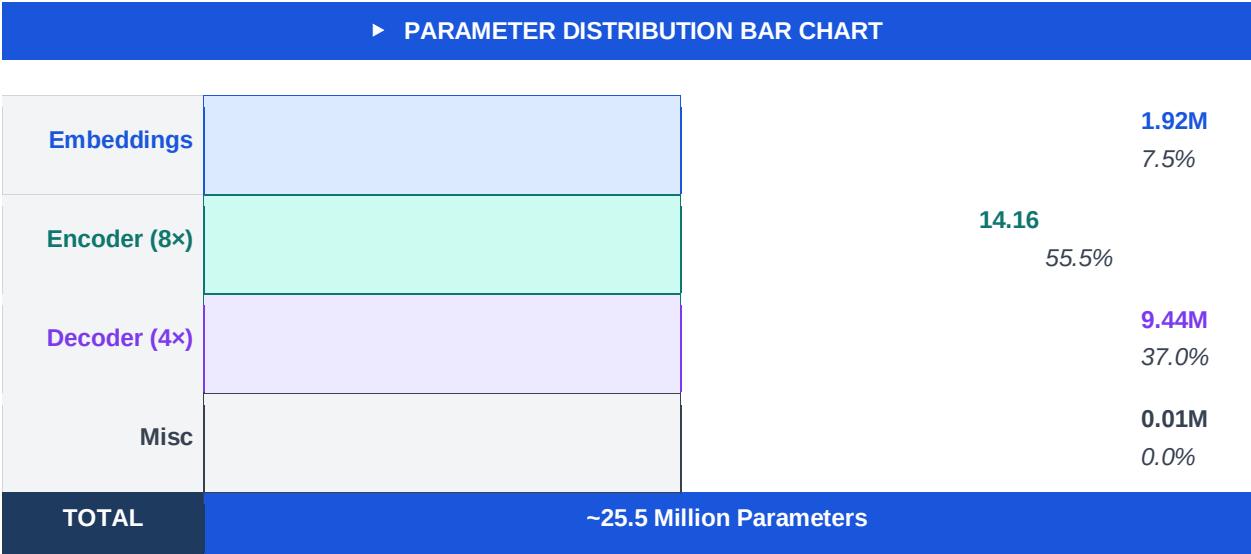
Standard transformers add absolute positional embeddings (fixed or learned positions) to every token. T5 and this model instead learn a relative position bias — the model learns how much attention to adjust based on the distance between two tokens ($i - j$), not their absolute positions.

► ABSOLUTE vs RELATIVE POSITION COMPARISON		
Absolute Positional Embedding Fixed sinusoidal or learned positions Position 1, 2, 3... tied to absolute index ✗ Poor generalization to unseen lengths	VS	Relative Position Bias (T5-style) Learned bias per relative offset ($i-j$) Attends to 'how far apart' not 'where' ✓ Better cross-language generalization

For translation: relative bias generalizes better across different sentence lengths and helps the model handle Gondi's free-ish word order more robustly.

9. Parameter Breakdown — Visual Summary

The ~25.5M parameters are distributed across four major components. The encoder dominates because deep semantic understanding of the source language is the primary bottleneck for low-resource translation.



10. Standard T5 vs This Custom Architecture

Understanding how this model differs from vanilla T5 highlights the deliberate design choices made for the Gondi translation task.

Aspect	Standard T5	This Custom ByT5-Inspired
Tokenizer	SentencePiece (32k vocab)	Custom BPE (~5k vocab, tri-lingual)
Encoder/Decoder	6/6 symmetric layers	8/4 asymmetric (heavier encoder)

Activation	ReLU	Gated-GeLU (better morphology)
Position Bias	Absolute embeddings	Relative position bias (better generalization)
Target	General English NLP	Gondi ↔ English/Hindi (low-resource)
Param Count	60M–11B	~25.5M (fits on consumer GPU)

11. Why This Architecture Works for Gondi

Design Choice	Why It Helps Gondi
Heavy Encoder (8 layers)	Gondi has complex agglutinative morphology. More layers = deeper understanding of roots and suffixes.
Light Decoder (4 layers)	Generation is simpler once the source is well-understood. Saves parameters for the encoder.
Gated-GeLU activation	Better gradient flow and information routing vs ReLU, proven on low-resource NLP benchmarks.
Custom BPE (5k vocab)	Small vocab reduces sparsity on a 60K corpus. BPE handles Gondi morpheme boundaries well.
Relative position bias	Gondi has more flexible word order than English; relative bias handles this better.
~25.5M total params	Avoids overfitting on 60K examples. A 100M+ model would memorize rather than generalize.

12. Final Conclusion

This architecture combines four key innovations:

- ✦ T5 Sequence-to-Sequence Learning — flexible text-in / text-out framework
- ✦ ByT5 Asymmetric Encoder-Decoder — deep understanding, fast generation
- ✦ Gated Feed-Forward Layers — better morphological reasoning
- ✦ Custom Multilingual BPE Tokenization — Gondi + Hindi + English

The result is a lightweight, research-oriented multilingual translation model purpose-built for an under-resourced language.